

MSc in Data Science

Data Analytics & Algorithms — CA1

FaceTrace

Project Logbook

Weekly iteration log, individual contributions, and AI/ML tool evaluation.

Field	Detail
Group Members	Anjaly · Surekha
Weekly cadence	5-minute presentation to the class group at each weekly slot
Total weekly entries	5 weeks (Week 5 is the final presentation)

Table of Contents

1. Logbook Overview & Methodology	3
2. Weekly Logbook Entries	4
3. Iteration Log — 8 Major Iterations.....	7
4. Critical Evaluation of AI/ML Tools Used	8
4.1 AI/ML tools used during development	8
5. Individual Contribution Summary	9
6. Reflection & Lessons Learned.....	10
6.1 What went well	10
6.2 What we would do differently	10
6.3 Topics learnt.....	10
6.4 Self-evaluation	10

1. Logbook Overview & Methodology

This logbook records the work undertaken by Anjaly and Surekha on the FaceTrace project across five weekly milestones, each closed by a five-minute presentation to the class group. Week 1 was the proposal and kick-off; Week 5 was the final presentation. The logbook is structured as follows. Section 2 provides the week-by-week entry, with the iteration focus, an itemised split of what each team member did that week, the practical impact (parameter changes, accuracy or latency observations, defects found and resolved), and the GitHub commit count. Section 3 consolidates the eight major iteration milestones referenced from the main report. Section 4 holds the GitHub commit history excerpt. Section 5 holds the team's evaluation of the AI/ML tools used during development this section is human-authored as required by the assessment rules. Section 6 summarises individual contributions across the project. Section 7 holds the team's reflection on what was learnt, what could have gone better, and what we would do differently next time.

2. Weekly Logbook Entries

Week 1

Week	Week 1
Iteration focus	Project proposal & kick-off. First 5-minute pitch to the class group.
Anjaly	Drafted the product definition and one-page proposal. Scoped the algorithm choice (CNN as a well-understood baseline). Set up the team's Trello board for week-by-week milestones.
Surekha	Drafted the slide for the kick-off pitch. Scoped the algorithm choice (CNN as a well-understood baseline).
Practical impact	Decisions: domain = privacy/identity (healthtech / regtech crossover); two-algorithm head-to-head comparison agreed. No code yet.

Week 2

Week	Week 2
Iteration focus	Repository bootstrap, Python environment, dataset acquisition, first EDA pass.
Anjaly	set up Python 3.11 + venv + pinned requirements.txt (NumPy, OpenCV-python, flask, pillow). Wrote project skeleton (app.py, search.py, db.py).
Surekha	Downloaded LFW Top-50 (~2,461 images, 50 identities). Wrote lfw.ipynb: per-identity image count histogram, image dimension consistency check, sample lighting/pose montage. Identified the long-tail bias (Bush dominates with 530 images).
Practical impact	Confirmed dataset integrity. Class imbalance noted in advance so it does not surprise us later. Project scaffolding now allows multi-file imports.

Week 3

Week	Week 3
Iteration focus	Iteration 2 basic UI, multipart upload pipeline, OpenCV face detection.
Anjaly	Implemented the Flask /api/search endpoint accepting multipart/form-data. Wrote the minimal HTML upload form and JavaScript fetch handler with help of claude. End-to-end I/O round-trip working: upload → backend → echo back filename.
Surekha	Integrated OpenCV Haar Cascade (haarcascade_frontalface_default.xml). Tuned scaleFactor (1.1 → 1.05) and neighbours (5 → 4) on a 100-image sample. Detection rate improved from ~78% to ~88%.
Practical impact	Pipeline accepts uploads correctly. Face detection rate of approximately 85–90% across LFW Top-50; ~10–15% fall back to using the full image. Documented as a known limitation.

Week 4

Week	Week 4
Iteration focus	Iterations 3, 4 & 5 CNN architecture from scratch, database indexing & cosine search, UI refinement.
Anjaly	Implemented ConvLayer (with stride and padding via im2col), BatchNormLayer, MaxPoolLayer, DenseLayer, DropoutLayer, ReLU activation, and L2 normalisation. Designed and implemented the Face Database class: NumPy. for the (N, 128) embedding matrix; JSON for parallel metadata; Rewrote frontend CSS for a dark luxury aesthetic with similarity-percentage badges and animated similarity bars.
Surekha	Implemented the preprocessing pipeline: Haar Cascade crop, 20% margin expansion, resize to 64×64, CLAHE contrast enhancement, [0, 255] → [0, 1] normalisation, HWC → CHW reorder. Indexed the LFW Top-50 dataset using the CNN — 2,461 embeddings written, total run time approximately 10 minutes on CPU. Implemented the threshold slider with debounced live re-ranking, the person-grouped results view, and the architecture information panel.
Practical impact	Four-block CNN producing 128-d L2-normalised embeddings; self-similarity verified at 1.000; inference time ~200ms per image on CPU. Full CNN search pipeline working end-to-end: upload → detect → embed → search → ranked results in <500ms. UI now comparable to a small commercial face-search tool.

Week 5 (Final)

Week	Week 5 (Final)
Iteration focus	Iterations 6, 7 & 8 — internet-wide search investigation & pivot, full ViT from scratch, comparison interface, final integration and submission.
Anjaly	Drafted a candidate web-crawler architecture and tested per-domain crawl rates against five public news archives. Studied the ViT paper (Dosovitskiy et al., 2020) and produced a teaching-quality whiteboard explanation of multi-head self-attention. Implemented PatchEmbedding (linear projection of $64 \text{ patches} \times 192 \rightarrow 128\text{-d}$) and MLPBlock (with GELU and $4\times$ hidden expansion). Stitched the four TransformerBlocks together with residual connections and layer norms in the right order. Built compare.html: parallel fetch to both /api/search endpoints; side-by-side result cards; per-criterion winner using weighted score = $0.4 \times \text{match_count} + 0.4 \times \text{top_similarity} + 0.2 \times (1/\text{latency})$. Final code review across both repos; cleaned up commented-out code; tightened error messages on the Flask endpoints. Wrote the README installation and run instructions. Recorded the demo screen capture for the final presentation.
Surekha	Researched the legal landscape: GDPR Articles 9 and 22, Irish Data Protection Act 2018, Terms of Service of major social platforms. Documented the six barriers (web-scale DB, legal, infrastructure, untrained models, JavaScript rendering, GDPR). Implemented MultiHeadSelfAttention (4 heads, scaled dot-product); tested 2, 4, and 8 heads on a 50-image sample (4 selected for the speed/expressivity trade-off on CPU); wrote LayerNorm from scratch. Implemented the [CLS] token, the learnable positional encoding, the final LayerNorm + Linear projection, and L2 normalisation. Deployed the ViT app on Flask port 5001 with the same /api/search contract as the CNN service. Wrote the architecture comparison table, the auto-generated textual conclusion, and the per-query winner badge; tested across 12 reference images covering different identities, poses, and lighting. End-to-end system testing on edge cases (no face detected, corrupt image upload, empty database, simultaneous CNN and ViT requests). Authored the slide deck mirroring the main report's section structure for the final presentation.
Practical impact	Pivot decision: abandoned internet-wide search and doubled down on dual-algorithm depth — six barriers became Section 6 of the main report. Caught and fixed an early shape bug in attention output: tensor was (heads, tokens, dim) when next layer expected (tokens, heads $\times$ dim). ViT produces 128-d embeddings; self-similarity confirmed at 1.000; inference time approximately 800ms per image ($4\times$ slower than CNN). Two complete working applications run side-by-side. Live CNN vs ViT demo now possible; auto winner detection correct on 11/12 reference queries. All 47 manual test cases pass. Final report and logbook reviewed by both team members and signed off; deck and screen recording submission-ready.

3. Iteration Log — 8 Major Iterations

The week-by-week entries above map onto eight major engineering iterations. The table below consolidates each iteration with its accuracy or latency impact, for easy reference at marking time.

Iteration	Week	Focus	Practical impact / metrics
1	Week 1	Dataset selection & project setup	LFW Top-50 chosen; class imbalance noted; environment & repo bootstrapped.
2	Week 2 – 3	Basic UI, upload pipeline, face detection	Detection rate $\approx$ 85–90% after Haar Cascade tuning; pipeline I/O verified.
3	Week 4	CNN architecture from scratch	128-d embeddings; self-similarity = 1.000; $\sim$ 200ms inference.
4	Week 4	Database indexing & cosine search	2,461 embeddings indexed; full search <500ms end-to-end.
5	Week 4	UI refinement	Dark theme, threshold slider, person grouping; commercial-grade UX.
6	Week 5	Internet-wide search investigation & pivot	Six barriers documented; project pivoted to dual-algorithm depth.
7	Week 5	ViT from scratch & deployment	128-d embeddings; self-similarity = 1.000; $\sim$ 800ms inference; second Flask app live on port 5001.
8	Week 5	Comparison interface	compare.html runs both apps; auto winner correct on 11/12 reference queries.

4. AI/ML Tools Used

4.1 AI/ML tools used during development

- Tool 1:. Claude (Anthropic) – claude was used for the entire project.
- Tool 2: claude cowork – cowork was used to build the entire UI .

5. Individual Contribution Summary

The week-by-week split is in Section 2; this section consolidates the cumulative contribution per team member. Both members presented at every weekly five-minute slot, alternating who led the slot.

Anjaly	Surekha
CNN architecture design and NumPy implementation (ConvLayer, BatchNorm, MaxPool, Dense, Dropout, ReLU, L2)	Dataset research, sourcing, and exploratory data analysis (LFW Top-50)
Flask REST API for the CNN service (port 5000)	Vision Transformer (ViT) implementation (PatchEmbedding, MHSA, MLP, TransformerBlock, CLS, positional encoding)
Face database design and cosine similarity search (NumPy .npy + JSON)	Face detection pipeline (OpenCV Haar Cascade, margin expansion, fallback handling)
LFW dataset indexing pipeline	Frontend UI design (CNN blue theme, ViT purple theme, responsive layout)
Comparison interface (compare.html, parallel-fetch logic, winner detection)	Flask REST API for the ViT service (port 5001)
ArcFace V2 future-work scoping; final code review	Weekly 5-minute presentations (lead in odd weeks); ongoing logbook documentation

6. Reflection & Lessons Learned

6.1 What went well

- Implementing both CNN and ViT from scratch in NumPy gave a much deeper understanding of what each layer actually does. We can now sketch a multi-head attention block on a whiteboard end-to-end, which we could not do at the start of the project.
- The decision to keep an identical preprocessing pipeline for both algorithms made the comparison results genuinely interpretable rather than confounded by data prep.
- Documenting the internet-wide search pivot openly (Section 6 of the main report) turned a dead-end attempt into a substantive analytical contribution.
- The five weekly five-minute presentations forced concrete weekly progress; without that cadence we would likely have missed the iteration discipline this project rewards.

6.2 What we would do differently

- Train the networks. The biggest single improvement would be to add a contrastive-loss training loop on labelled face pairs. We chose to keep weights random for time reasons, but the resulting embeddings are not semantically meaningful — this is the limitation that most affects the report.
- Replace the brute-force cosine search with a vector index (FAISS) once the dataset exceeds about 50,000 images. The current approach is fine for LFW Top-50 but does not scale.
- Replace Haar Cascade with a modern detector (RetinaFace, MTCNN). Haar Cascade fails on profile shots, and our 10–15% fallback rate is the dominant source of low-quality results.
- Add formal accuracy metrics. With trained weights, TAR@FAR and rank-1 identification rate become meaningful and would let us quantify the CNN/ViT gap rather than describe it qualitatively.

6.3 Topics learnt

- Multi-head self-attention end-to-end: scaled dot-product, masking, GELU, residual connections, the role of LayerNorm before vs after the attention block.
- Convolution from first principles: im2col, BatchNorm running statistics, MaxPool back-propagation.
- L2 normalisation as the bridge between an embedding network and cosine similarity.
- How to scope an academic project against a real-world commercial system (PimEyes) without overreaching, and how to defend the scope decision in writing.
- Pair-programming and weekly stand-up cadence as project-management primitives.

6.4 Self-evaluation

Both team members met their committed contributions on time across all five weekly milestones. The five-minute presentations were delivered every week without exception. The split between us was even: Anjaly led the CNN side and the search/indexing infrastructure; Surekha led the ViT side, the data pipeline, and

the frontend. Neither of us depended on the other for understanding both can explain both architectures end-to-end at whiteboard level.